Add icon Add cover

3. Use Least Privilege RBAC

📌 Below is a **structured, execution-ready guide** tailored for how you operate (SOP/playbook style), broken into 3 parts exactly as requested:

✅ 1) How to Create a Custom Azure RBAC Role for Pipelines

(Practical + teachable + implementation-focused)

🔷 Concept (Why you do this)

Azure RBAC roles are simply **collections of allowed actions** tied to a **scope**.

Custom roles exist because:

> Built-in roles are often too broad; custom roles let you define exactly what is allowed.

🔷 Step-by-Step Process (SOP)

Step 1 — Identify EXACT required actions

Start with:

- What resources are deployed?
- What actions are performed (read, write, delete)?

Microsoft guidance: determine permissions by identifying **resource providers and operations**.

✅ Example:

- Deploy ARM/Bicep → `Microsoft.Resources/deployments/*`
- Deploy Web App → `Microsoft.Web/sites/*`
- Read resources → `Microsoft.Resources/*/read`

Step 2 — Start from a built-in role (baseline)

Use:

- Contributor → if you need broad resource control
- Reader → if read-only

Then **reduce permissions**, not expand

(best practice per Microsoft guidance on using existing roles as a baseline).

Step 3 — Define the custom role JSON

Azure roles are JSON definitions with:

- `Actions` (allowed operations)
- `NotActions` (explicit exclusions)
- `DataActions` (data-level permissions)
- `AssignableScopes` (where it applies)

Step 4 — Create the role (CLI example)

```shell
az role definition create --role-definition role.json
```

(Command syntax based on standard Azure RBAC tooling; step itself not explicitly detailed in sources, but aligned to documented methods for creation via CLI/PowerShell/Portal.)

Step 5 — Assign the role to the pipeline identity

Example (service principal):

```shell
az role assignment create \
  --assignee <SERVICE_PRINCIPAL_ID> \
  --role "Pipeline Deployment Operator" \
  --scope /subscriptions/<SUB_ID>/resourceGroups/<RG_NAME>

```

🔷 Example: Minimal Deployment Role (BEST Practice)

```json
{
  "Name": "Pipeline Deployment Operator",
  "IsCustom": true,
  "Description": "Allows deployment of resources but prevents RBAC changes",
  "Actions": [
    "Microsoft.Resources/deployments/*",
    "Microsoft.Resources/subscriptions/resourceGroups/read"
  ],
  "NotActions": [
    "Microsoft.Authorization/*"
  ],
  "AssignableScopes": [
    "/subscriptions/<SUB_ID>/resourceGroups/<RG_NAME>"
  ]
}

```

🔷 Design Principles (What makes it “BEST”)

- ✅ Only required **Actions**
- ✅ Explicitly deny RBAC changes (`Microsoft.Authorization`)
- ✅ Scoped to **resource group**
- ✅ No subscription-level access

✅ 2) Least Privilege Best Practices for GitHub Actions (CI/CD)

🔷 Core Principle

Pipeline identity = **short-lived + narrowly scoped + purpose-specific**

Microsoft guidance highlights:

- Use identity provider integration (Entra ID)
- Apply **principle of least privilege across all role assignments**

🔷 Best Practices (Condensed)

1. Use Federated Identity (OIDC) — NOT secrets

GitHub docs + Azure guidance:

- Use **OIDC tokens instead of stored credentials**
- Tokens are short-lived and automatically issued per run

✅ Outcome:

- No secrets stored
- No credential rotation risk

---

2. Assign RBAC roles at the lowest scope

Scopes hierarchy:

- Management group → Subscription → Resource group → Resource

Least privilege = smallest possible scope

✅ Always prefer:

- Resource group over subscription

---

3. Never give pipelines “Owner”

Reason:

- Owner includes **permission to assign roles**

Risk:

- Pipeline can escalate privileges → full compromise

---

4. Avoid “Contribute everywhere” model

Common failure:

> Contributor at subscription = hidden Owner-lite risk

Better:

- Split roles by environment:
  - Dev pipeline → Dev RG
  - Prod pipeline → Prod RG
- Or use separate service principals

---

5. Restrict what pipelines can execute

Service connection risk:

- Pipelines can execute arbitrary scripts with access to resources

✅ Controls:

- Branch protection
- PR reviews
- Environment approvals

6. Separate identities per environment

Example:

- `pipeline-dev-sp`
- `pipeline-prod-sp`

✅ prevents lateral movement across environments

---

7. Monitor and audit usage

- Log role assignments
- Track token usage
- Review regularly

Microsoft guidance:

> periodic access reviews help maintain least privilege

🔷 "Gold Standard" GitHub Actions Model

	Layer	Best Practice
1	Authentication	OIDC federation
2	Identity	Service principal per env
3	RBAC	Custom role
4	Scope	Resource group
5	Execution	PR + approval controls
6	Auditing	Logs + reviews

✅ 3) Checklist for Securing Microsoft 365 Pipeline Identities

(This applies to automation using Microsoft Graph, PowerShell, or apps)

🔷 Identity & Access

☐ Use **App Registration (Service Principal)** for automation

☐ Prefer **delegated permissions** where possible (safer scope)

☐ Use **application permissions only when required**

Microsoft Graph guidance:

> request the least privileged permissions your app needs

🔷 Permission Management

☐ Minimise Graph permissions (e.g. Read vs ReadWrite)

☐ Avoid tenant-wide permissions unless absolutely necessary

☐ Review all permissions before granting admin consent

⚠️ Microsoft warns:

> admin consent can grant access to significant organisational data

---

🔷 Administrative Role Control

☐ Avoid using **Global Administrator** for pipelines

☐ Use service-specific roles instead (Exchange, Teams, SharePoint)

☐ Limit number of privileged roles

Microsoft guidance:

> use roles with the fewest permissions and limit admin access

---

🔷 Privileged Access Management (Critical)

☐ Enable **Privileged Identity Management (PIM)**

☐ Use **Just-in-Time role activation**

☐ Require **MFA for role activation**

Microsoft best practice:

- Least privilege = permissions + scope + time
- Use PIM for temporary access

🔷 Application & Automation Security

☐ Avoid long-lived client secrets

☐ Use **certificates or federation** instead

☐ Rotate credentials regularly (if used)

☐ Restrict app login to specific users/groups

🔷 Monitoring & Governance

☐ Enable audit logs for:

- Role assignments
- App consent
- Sign-ins

☐ Conduct:

- Regular access reviews
- App permission audits

🔷 Secure-by-Design Model (What you want)

```shell
Pipeline/App Identity
      ↓
Scoped Permissions (least privilege)
      ↓
Time-bound Access (PIM / tokens)
      ↓
Monitored + Audited Usage

```

🔥 Final Executive Summary (What matters most)

Azure (Pipelines)

- Avoid **Owner**
- Scope to **resource groups**
- Use **custom roles**
- Use **OIDC federation**

GitHub Actions

- No secrets → use federation
- No shared identities
- No broad RBAC
- Control **execution path**

Microsoft 365

- Minimise Graph permissions
- Limit admin roles
- Use **PIM + JIT access**
- Audit everything

If you want next:

I can convert this into a **formal SOP + policy document + training slides** for your team (aligned to ISO 27001 / ISO 9001 / DevSecOps standards).

Below is a **structured, execution-ready guide** tailored for how you operate (SOP/playbook style), broken into 3 parts exactly as requested:

✅ 1) How to Create a Custom Azure RBAC Role for Pipelines

(Practical + teachable + implementation-focused)

🔷 Concept (Why you do this)

Azure RBAC roles are simply **collections of allowed actions** tied to a **scope**.

Custom roles exist because:

> Built-in roles are often too broad; custom roles let you define exactly what is allowed.

🔷 Step-by-Step Process (SOP)

Step 1 — Identify EXACT required actions

Start with:

- What resources are deployed?
- What actions are performed (read, write, delete)?

Microsoft guidance: determine permissions by identifying **resource providers and operations**.

✅ Example:

- Deploy ARM/Bicep → `Microsoft.Resources/deployments/*`
- Deploy Web App → `Microsoft.Web/sites/*`
- Read resources → `Microsoft.Resources/*/read`

Step 2 — Start from a built-in role (baseline)

Use:

- Contributor → if you need broad resource control
- Reader → if read-only

Then **reduce permissions**, not expand

(best practice per Microsoft guidance on using existing roles as a baseline).

Step 3 — Define the custom role JSON

Azure roles are JSON definitions with:

- `Actions` (allowed operations)
- `NotActions` (explicit exclusions)
- `DataActions` (data-level permissions)
- `AssignableScopes` (where it applies)

Step 4 — Create the role (CLI example)

Shell

```
az role definition create --role-definition role.json
```

(Command syntax based on standard Azure RBAC tooling; step itself not explicitly detailed in sources, but aligned to documented methods for creation via CLI/PowerShell/Portal.)

Step 5 — Assign the role to the pipeline identity

Example (service principal):

Shell

```
az role assignment create \
  --assignee <SERVICE_PRINCIPAL_ID> \
  --role "Pipeline Deployment Operator" \
  --scope /subscriptions/<SUB_ID>/resourceGroups/<RG_NAME>

```

🔷 Example: Minimal Deployment Role (BEST Practice)

JSON

```
{
  "Name": "Pipeline Deployment Operator",
  "IsCustom": true,
  "Description": "Allows deployment of resources but prevents RBAC changes",
  "Actions": [
    "Microsoft.Resources/deployments/*",
    "Microsoft.Resources/subscriptions/resourceGroups/read"
  ],
  "NotActions": [
    "Microsoft.Authorization/*"
  ],
  "AssignableScopes": [
    "/subscriptions/<SUB_ID>/resourceGroups/<RG_NAME>"
  ]
}

```

🔷 Design Principles (What makes it "BEST")

- ✅ Only required **Actions**
- ✅ Explicitly deny RBAC changes (`Microsoft.Authorization`)
- ✅ Scoped to **resource group**
- ✅ No subscription-level access

✅ 2) Least Privilege Best Practices for GitHub Actions (CI/CD)

🔷 Core Principle

Pipeline identity = **short-lived + narrowly scoped + purpose-specific**

Microsoft guidance highlights:

- Use identity provider integration (Entra ID)
- Apply **principle of least privilege across all role assignments**

🔷 Best Practices (Condensed)

1. Use Federated Identity (OIDC) — NOT secrets

GitHub docs + Azure guidance:

- Use **OIDC tokens instead of stored credentials**
- Tokens are short-lived and automatically issued per run

✅ Outcome:

- No secrets stored
- No credential rotation risk

---

2. Assign RBAC roles at the lowest scope

Scopes hierarchy:

- Management group → Subscription → Resource group → Resource

Least privilege = smallest possible scope

✅ Always prefer:

- Resource group over subscription

---

3. Never give pipelines “Owner”

Reason:

- Owner includes **permission to assign roles**

Risk:

- Pipeline can escalate privileges → full compromise

---

4. Avoid “Contribute everywhere” model

Common failure:

> Contributor at subscription = hidden Owner-lite risk

Better:

- Split roles by environment:
  - Dev pipeline → Dev RG
  - Prod pipeline → Prod RG
- Or use separate service principals

---

5. Restrict what pipelines can execute

Service connection risk:

- Pipelines can execute arbitrary scripts with access to resources

✅ Controls:

- Branch protection
- PR reviews
- Environment approvals

---

6. Separate identities per environment

Example:

- `pipeline-dev-sp`
- `pipeline-prod-sp`

✅ prevents lateral movement across environments

---

7. Monitor and audit usage

- Log role assignments
- Track token usage
- Review regularly

Microsoft guidance:

> periodic access reviews help maintain least privilege

🔷 “Gold Standard” GitHub Actions Model

	Layer	Best Practice
1	Authentication	OIDC federation
2	Identity	Service principal per env
3	RBAC	Custom role
4	Scope	Resource group
5	Execution	PR + approval controls
6	Auditing	Logs + reviews

✅ 3) Checklist for Securing Microsoft 365 Pipeline Identities

(This applies to automation using Microsoft Graph, PowerShell, or apps)

🔷 Identity & Access

☐ Use **App Registration (Service Principal)** for automation

☐ Prefer **delegated permissions** where possible (safer scope)

☐ Use **application permissions only when required**

Microsoft Graph guidance:

> request the least privileged permissions your app needs

🔷 Permission Management

☐ Minimise Graph permissions (e.g. Read vs ReadWrite)

☐ Avoid tenant-wide permissions unless absolutely necessary

☐ Review all permissions before granting admin consent

⚠️ Microsoft warns:

> admin consent can grant access to significant organisational data

---

🔷 Administrative Role Control

☐ Avoid using **Global Administrator** for pipelines

☐ Use service-specific roles instead (Exchange, Teams, SharePoint)

☐ Limit number of privileged roles

Microsoft guidance:

> use roles with the fewest permissions and limit admin access

---

🔷 Privileged Access Management (Critical)

☐ Enable **Privileged Identity Management (PIM)**

☐ Use **Just-in-Time role activation**

☐ Require **MFA for role activation**

Microsoft best practice:

- Least privilege = permissions + scope + time
- Use PIM for temporary access

🔷 Application & Automation Security

☐ Avoid long-lived client secrets

☐ Use **certificates or federation** instead

☐ Rotate credentials regularly (if used)

☐ Restrict app login to specific users/groups

🔷 Monitoring & Governance

☐ Enable audit logs for:

- Role assignments
- App consent
- Sign-ins

☐ Conduct:

- Regular access reviews
- App permission audits

🔷 Secure-by-Design Model (What you want)

Shell

```
Pipeline/App Identity
      ↓
Scoped Permissions (least privilege)
      ↓
Time-bound Access (PIM / tokens)
      ↓
Monitored + Audited Usage

```

🔥 Final Executive Summary (What matters most)

Azure (Pipelines)

- Avoid **Owner**
- Scope to **resource groups**
- Use **custom roles**
- Use **OIDC federation**

GitHub Actions

- No secrets → use federation

- No shared identities
- No broad RBAC
- Control **execution path**

Microsoft 365

- Minimise Graph permissions
- Limit admin roles
- Use **PIM + JIT access**
- Audit everything

If you want next:

I can convert this into a **formal SOP + policy document + training slides** for your team (aligned to ISO 27001 / ISO 9001 / DevSecOps standards).